

Bug Report — HW06 API Testing (§6.5)

- **Sinh viên:** Lê Nhật Duy — **MSSV:** 23127178
- **SUT:** EShop — <https://github.com/ttbhanh/eshop-sut> · fork nhóm: [DuyITL0R/group05_eshop](#) commit `f0f3b7b`
- **Môi trường:** `localhost:3000` · Node `v22.23.1` · Newman `6.2.2` · macOS `26.1` arm64
- **Tái hiện lại toàn bộ:** `bash bug-report/verify-bugs.sh` → log lượt chạy thật: [verify-bugs-output.txt](#)
- **Trạng thái:** **22 bug xác nhận** (19 từ bộ test AI + 3 từ case sinh viên chọn) + **2 rủi ro** + **1 câu hỏi nghiệp vụ**. Issues: [#323–#341](#) và [#402–#407](#) (không có bug nào chỉ suy từ đọc code).
- **GitHub Issues:** [#323–#341](#) (19 bug từ bộ AI) và [#402–#407](#) (6 bug do case sinh viên chọn) trên [DuyITL0R/group05_eshop](#) — mỗi issue có đủ 8 trường của template, ảnh báo cáo Newman nhúng sẵn, và lệnh tái hiện. Script tạo: [create-github-issues.sh](#) · nội dung: [issues/](#)

Luật của file này: một dòng chỉ được gọi là bug khi (1) tái hiện được bằng request thật, (2) có test case trong collection bắt được nó, (3) expected có **căn cứ từ spec/FR/SEC**, không phải suy đoán. Cột "Bằng chứng" trở tới đúng mục trong log của `verify-bugs.sh`.

Bảng tổng hợp

#	Module	Bug	Yêu cầu vi phạm	Severity / Priority	Test case bắt được	Issue
BUG-01	products/search	SQL injection qua <code>search</code> — payload vô hiệu hoá điều kiện <code>WHERE</code> , trả toàn bộ bảng	SEC-05	Critical / P1	TC-PRODLIST-024, 025, 026, 027	#323
BUG-02	products/search	Lỗi CSDL trả HTML kèm <code>SQLITE_ERROR</code> (rò rỉ thông tin nội bộ, sai schema)	SEC-05 · spec §3.1	High / P1	TC-PRODLIST-026, 104, 105	#324
BUG-03	products/detail	<code>GET /api/products/:id</code> không tồn tại / sai kiểu → 200 {} thay vì 404	spec §3.2	Medium / P2	TC-PRODLIST-023, 033, 034, 035, 036	#325
BUG-04	products/detail	<code>price</code> trả về string khi <code>id</code> chẵn, number khi <code>id</code> lẻ	spec §3.2/ §3.3	Medium / P2	TC-PRODLIST-107	#326
BUG-05	products/search	Tìm kiếm tiếng Việt có dấu phân biệt hoa/thường (<code>áo</code> → 0 dòng, <code>Áo</code> → 1 dòng)	FR-05	Medium / P2	TC-PRODLIST-101	#327
BUG-06	products/search	Ký tự <code>%</code> và <code>_</code> của <code>LIKE</code> không được escape → input dùng như pattern	FR-05 · SEC-05	Medium / P2	TC-PRODLIST-103, 102	#328

#	Module	Bug	Yêu cầu vi phạm	Severity / Priority	Test case bắt được	Issue
BUG-07	cart	POST /api/cart không validate field nào: quantity 0/âm/thập phân/sai kiểu, body rỗng	FR-07	High / P1	TC-CART-003...015, 107	#329
BUG-08	cart	Price tampering — client gửi price=1 cho sản phẩm 200.000đ và giá đó vào giỏ	FR-07 · FR-08	Critical / P1	TC-CART-025, 101, 102	#330
BUG-09	cart / checkout	Giỏ hàng không được xoá sau checkout → bấm lại tạo đơn trùng	FR-07 · FR-08	High / P1	TC-CART-029, 103	#331
BUG-10	cart	Mass assignment : field lạ (role , isAdmin) được lưu nguyên vào giỏ	SEC-06	Medium / P2	TC-CART-104	#332
BUG-11	cart	Thêm được sản phẩm không tồn tại / đã bị xoá vào giỏ	FR-07	Medium / P2	TC-CART-010, 011, 012, 106	#333
BUG-12	cart	Cùng một sản phẩm tạo nhiều dòng , không cộng dồn số lượng	FR-07	Low / P3	TC-CART-027	#334
BUG-13	products/admin	PUT / POST / DELETE /api/products không có tầng xác thực nào — khách và user thường sửa/tạo/xoá được sản phẩm	SEC-02, SEC-03	Critical / P1	TC-PRODUPD-031...034, 102, 103, 107, 108	#335
BUG-14	products/admin + detail	DoS : khách không đăng nhập làm sập cả backend bằng 2 request	SEC-02 · độ tin cậy	Critical / P1	(tái hiện riêng — xem §BUG-14)	#336
BUG-15	products/admin	Partial update ghi NULL đè dữ liệu cũ nhưng vẫn trả 200 Product updated	FR-15	High / P1	TC-PRODUPD-105	#337
BUG-16	products/admin	Không validate price (âm, 0, chuỗi), name rỗng, category_id không tồn tại	FR-15 · đề §6.1 (price > 0)	High / P2	TC-PRODUPD-003, 004, 009...012, 016...018	#338
BUG-17	products/admin	PUT vào :id không tồn tại vẫn trả 200 Product updated	spec §3.3	Medium / P2	TC-PRODUPD-020...024, 029	#339
BUG-18	products/admin	Mất chính xác số tiền > 2^53 (9007199254740993 → ...992)	FR-15	Low / P3	TC-PRODUPD-014	#340
BUG-19	users (ngoài phạm vi)	GET /api/users/me trả mật khẩu plaintext	SEC-01	Critical / P1	(phát hiện khi dựng setup — xem §BUG-19)	#341

Case sinh viên chọn (§6.3) — 3 bug xác nhận + 2 rủi ro + 1 câu hỏi nghiệp vụ

Sau khi soát lại lần năm, 3 trong 6 mục ban đầu bị hạ khỏi mức "bug". Lý do ghi thẳng ở đây vì nó là điều đáng đọc nhất của phần này: một phát hiện chỉ được gọi là bug khi expected có **căn cứ bắt buộc**, và ba mục dưới đây không có.

#	Module	Nội dung	Yêu cầu	Severity	Case	Issue
BUG-23	cart / users	Token của user đã bị xoá vẫn mở được giỏ — <code>authenticateToken</code> chỉ verify chữ ký, không đối chiếu bằng <code>users</code> (<code>server.js:104–110</code>); <code>jwt.sign</code> cũng không có <code>expiresIn</code> nên token không bao giờ hết hạn	SEC-02	High / P1	TC-CART-208, 209	#405
BUG-24	products / admin	<code>imageUrl = javascript:alert(1)</code> được lưu nguyên. Không phải vấn đề escape (SEC-04 nói escape khi <i>hiển thị</i>) mà là thiếu validate scheme : spec §3.3 định nghĩa <code>imageUrl</code> dạng <code>http://...</code> , và một URL <code>javascript:</code> trong <code>href / src</code> chạy được kể cả khi đã escape đúng	spec §3.3 · SEC-04	Medium / P2	TC-PROD UPD-201, 202	#406
BUG-25	products / admin	<code>category_id</code> trò danh mục đã bị xoá vẫn ghi được — SQLite không bật <code>PRAGMA foreign_keys</code> , tầng API cũng không kiểm	FR-14 · spec §3.4	Medium / P2	TC-PROD UPD-205, 206	#407

Hạ mức: 2 rủi ro và 1 câu hỏi nghiệp vụ — không báo là bug

#	Nội dung	Vì sao KHÔNG phải bug	Case	Issue (đã sửa nhân)
R-01	GET <code>/api/products</code> không có phân trang; <code>?limit / ?page</code> bị bỏ qua, luôn trả toàn bộ bảng	Spec §3.1 chỉ định nghĩa <code>search</code> . Bỏ qua query param lạ là hành vi HTTP bình thường; không yêu cầu nào trong spec/FR đòi phân trang. Rủi ro hiệu năng là thật (DB thật của SUT ở HW05 có ~900k dòng) nhưng thuộc đề xuất cải tiến	TC-PRODLIST-201, 202 (<i>giờ xanh — characterization test</i>)	#402
R-02	Payload <code><script></code> được lưu và trả lại nguyên văn trong giỏ	SEC-04 nguyên văn : " <i>Mọi dữ liệu từ user nhập vào khi hiển thị trên UI phải được escape đúng cách, không dùng <code>innerHTML</code></i> ". Nó nói escape khi hiển thị , không cấm lưu. Tầng API trả <code>application/json</code> — payload là dữ liệu . Rủi ro hiện thực hoá ở UI, ngoài phạm vi bài API	TC-CART-204, 205 (<i>giờ xanh</i>)	#404
Q-01	Giỏ giữ giá lúc thêm, không cập nhật khi catalog đổi giá	Price-snapshot là chính sách hợp lệ và phổ biến (giá chốt lúc thêm giỏ). Spec §4.1/§4.2 không nói giỏ tham chiếu hay chụp giá → không suy ra được bên nào đúng. Cần quyết định nghiệp vụ , và nếu là chính sách thì phải công bố cho người mua	TC-CART-201→203 (<i>giờ xanh</i>)	#403

Rủi ro thật về tiền vẫn còn, nhưng nó nằm ở **BUG-08 (price tampering)** — chỗ đó client **tự đặt giá**, và căn cứ FR-07/FR-08 rõ ràng hơn nhiều so với Q-01.

Phân bố: 5 Critical · 6 High · 9 Medium · 2 Low = **22 bug**, cộng **2 rủi ro (R-01, R-02)** và **1 câu hỏi nghiệp vụ (Q-01)** được ghi riêng. 12/19 bug thuộc đúng 3 API được giao; 7 bug còn lại nằm ở **endpoint hỗ trợ** trong cùng chuỗi test (GET /api/products/:id , POST /api/checkout , POST /DELETE /api/products , GET /api/users/me).

BUG-14 — [BUG][products] Khách không đăng nhập làm sập toàn bộ backend bằng 2 request

Bug đáng chú ý nhất của bài: nó là **chuỗi ba lỗi** mà từng lỗi riêng lẻ chỉ ở mức Medium/High.

- **Found by:** không phải một test case đơn lẻ — phát hiện khi **BUG-13 + BUG-15 + BUG-04 gặp nhau** trong lúc chạy probe. Tái hiện tự động: `bash bug-report/verify-bugs.sh 14`
- **Requirement:** SEC-02 (API bảo mật đòi JWT) · FR-15 · yêu cầu phi chức năng về độ tin cậy
- **Severity / Priority:** **Critical / P1** — mất toàn bộ dịch vụ, kẻ tấn công **không cần tài khoản**
- **Environment:** `localhost:3000` · commit `f0f3b7b` · Node `v22.23.1`

Steps to reproduce (không có `Authorization` ở bất kỳ bước nào):

```
# 1. Ghi NULL vào price của một sản phẩm có id CHẴN (BUG-13 cho phép không cần token,
#   BUG-15 làm các field thiếu thành NULL)
curl -X PUT http://localhost:3000/api/products/18 \
  -H 'Content-Type: application/json' -d '{"name":"anything"}'
# → 200 {"message":"Product updated"}

# 2. Đọc chi tiết chính sản phẩm đó
curl http://localhost:3000/api/products/18
# → không có phản hồi; tiến trình node đã chết

# 3. Xác nhận toàn hệ thống đã ngừng
curl http://localhost:3000/api/products      # → connection refused
```

Expected: bước 1 trả **401** (SEC-02). Ngay cả khi bước 1 được phép, bước 2 phải trả JSON hợp lệ — một giá trị `null` trong CSDL không được làm sập tiến trình.

Actual: `server.js:161-162` chạy `if (row.id % 2 === 0) row.price = row.price.toString();` → `TypeError: Cannot read properties of null (reading 'toString')` **bên trong callback của sqlite3**, nằm ngoài mọi `try/catch` và ngoài middleware lỗi của Express → Node kết thúc tiến trình. Hệ thống chỉ sống lại khi có người khởi động lại thủ công, và **kẻ tấn công lặp lại được ngay** sau mỗi lần restart.

Ba lỗi hợp thành:

Mất	Lỗi	File
1	route PUT không gắn <code>authenticateToken</code> → không cần token	<code>server.js:179</code>
2	field thiếu trong body → ghi <code>NULL</code> đè	<code>server.js:180-188</code>
3	<code>price.toString()</code> không kiểm null, chạy trong callback <code>sqlite3</code>	<code>server.js:161-162</code>

Evidence: `verify-bugs-output.txt` mục BUG-14, có nguyên văn stack trace lấy từ `.run-logs/sut.log`.

Vì sao chuỗi này KHÔNG nằm trong collection Postman: một lượt Newman chạm vào nó sẽ làm SUT chết giữa đường và mọi case phía sau đó **vì môi trường**, không vì bug — tức phá luôn giá trị chứng minh của cả lượt chạy. Vì vậy `00-setup` của API-03 chọn **id lẻ** làm fixture, và chuỗi này được tái hiện riêng bằng script có khởi động lại SUT. Chi tiết lập luận: `test-cases/api-03-product-update/audit.md`.

BUG-13 — [BUG][products] PUT/POST/DELETE /api/products không có tầng xác thực nào

- **Found by test case:** TC-PRODUPD-031 (không token), TC-PRODUPD-032/102/103 (token user thường), TC-PRODUPD-107 (`POST`), TC-PRODUPD-108 (`DELETE`)
- **Requirement:** **SEC-02** (API có tính bảo mật phải yêu cầu JWT hợp lệ) và **SEC-03** (API Admin phải kiểm tra `role='admin'` , không chỉ kiểm tra sự tồn tại của token); spec §3.3 ghi rõ (Dành cho Admin)
- **Severity / Priority:** Critical / P1

Steps:

```
curl -X PUT http://localhost:3000/api/products/7 \
  -H 'Content-Type: application/json' \
  -d '{"name":"HACKED-anon","price":1,"description":"d","imageUrl":"u","category_id":1}'
curl http://localhost:3000/api/products/7      # name = "HACKED-anon" → dữ liệu ĐÃ đổi thật
```

Expected: 401 khi không có token; 403 khi token có `role='user'` . **Actual:** cả hai đều **200** và dữ liệu bị ghi. `POST /api/products` (tạo) và `DELETE /api/products/:id` (xóa) cũng vậy — so sánh: `POST /api/categories` (`server.js:249`) có `authenticateToken` , nên đây là **thiếu sót cục bộ ở 3 route**, không phải thiết kế toàn cục.

Điểm quan trọng của cách viết test case này: không dừng ở status code. TC-103 `GET` lại sản phẩm để chứng minh **dữ liệu đổi thật** — đó là khác biệt giữa "API trả sai mã lỗi" (Medium) và "người lạ sửa được catalog" (Critical).

Evidence: `verify-bugs-output.txt` mục BUG-13 · `reports/newman/*api-03*.html` folder `30-security-auth`

BUG-08 — [BUG][cart] Price tampering: giá trong giỏ do client quyết định

- **Found by test case:** TC-CART-101 (gửi `price=1`), TC-CART-102 (đọc lại giỏ để chứng minh), TC-CART-025
- **Requirement:** FR-07 (giỏ phản ánh sản phẩm thật) + FR-08 (tính tiền đơn hàng từ giỏ)
- **Severity / Priority:** Critical / P1 — ảnh hưởng trực tiếp doanh thu

Steps:

```
TOKEN=... # token user thường
curl -X POST http://localhost:3000/api/cart -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"id":7,"name":"HW06-Verify","price":1,"quantity":1}' # sản phẩm giá 200000
curl http://localhost:3000/api/cart -H "Authorization: Bearer $TOKEN"
# → [{"id":7,"name":"HW06-Verify","price":1,"quantity":1}]
```

Expected: giá trong giỏ phải bằng giá trong bảng products (200.000đ), hoặc request bị từ chối. **Actual:** server.js:290-295 là userCarts[userId].push(req.body) — nhận nguyên body, không đổi chiều catalog. Giá 1 đồng nằm trong giỏ và POST /api/checkout cũng nhận total_amount từ client.

Ghi chú trung thực về căn cứ: spec §4.2 có ghi price trong body, nên đọc thuần câu chữ thì gửi giá là đúng đặc tả. Bug được kết luận dựa trên **FR-07 + FR-08**: nếu client đặt giá thì client đặt luôn số tiền phải trả. Lập luận này được ghi nguyên văn trong test-cases/api-02-cart-add/audit.md để người chấm tự đánh giá.

BUG-01 — [BUG][products] SQL injection qua tham số search

- **Found by test case:** TC-PRODLIST-024 (%' OR '1'='1), 025 (' OR 1=1--), 026 (UNION), 027 (stacked)
- **Requirement: SEC-05** — Truy vấn CSDL phải dùng Parameterized Query, không nối chuỗi trực tiếp
- **Severity / Priority: Critical / P1**

Steps:

```
curl 'http://localhost:3000/api/products?search=Laptop' # → 2 dòng
curl -G --data-urlencode "search=%' OR '1'='1" \
  http://localhost:3000/api/products # → TOÀN BỘ bảng
curl -G --data-urlencode "search='" http://localhost:3000/api/products
# → HTTP 500, text/html: <h1>Database Error</h1><p>SQLITE_ERROR: unrecognized token: "'"</p>
curl -G --data-urlencode "search=' UNION SELECT 1,2,3,4,5--" \
  http://localhost:3000/api/products
# → 500 + "SELECTs to the left and right of UNION do not have the same number of result columns"
# ← câu này TIẾT LỘ số cột thật của bảng cho kẻ tấn công
```

Expected: search là giá trị, không phải mã SQL → 0 dòng, status 200. **Actual:** server.js:143 nối chuỗi: const query = `SELECT \* FROM products WHERE name LIKE '%\${searchQuery}%'`. Điều kiện WHERE bị vô hiệu hoá, và thông báo lỗi của engine bị trả về nguyên văn (→ BUG-02).

Kiểm tác động, không chỉ status: TC-PRODLIST-106 chạy sau payload '; DROP TABLE products-- để xác nhận bảng còn nguyên — db.all() của sqlite3 chỉ thực thi câu đầu, nên payload này không xoá được bảng. Ghi lại đúng như vậy: mức khai thác thực tế là **đọc dữ liệu + dò cấu trúc**, không phải phá dữ liệu.

BUG-15 — [BUG][products] Partial update ghi NULL đề dữ liệu, vẫn báo thành công

- **Found by test case:** TC-PRODUPD-104 (PUT chỉ có `name`) → TC-PRODUPD-105 (đọc lại, 3 assertion đồ)
- **Requirement:** FR-15 · **Severity / Priority:** High / P1

```
curl -X PUT localhost:3000/api/products/7 -H 'Content-Type: application/json' \
  -d '{"name":"HW06-Full","price":123456,"description":"desc","imageUrl":"u","category_id":1}'
curl -X PUT localhost:3000/api/products/7 -H 'Content-Type: application/json' \
  -d '{"name":"HW06-OnlyName"}'          # → 200 {"message":"Product updated"}
curl localhost:3000/api/products/7
# → {"id":7,"name":"HW06-OnlyName","price":null,"description":null,"imageUrl":null,"category_id":null}
```

Expected: 400 (đòi đủ field) hoặc 200 nhưng giữ nguyên các field không gửi. **Actual:** `server.js:180-188` luôn `SET name=?, price=?, description=?, imageUrl=?, category_id=?; field undefined` → `sqlite3` ghi `NULL`. Mất dữ liệu im lặng — response nói "Product updated". Đây cũng là **mắt thứ hai của BUG-14**.

BUG-05 — [BUG][products] Tìm kiếm tiếng Việt có dấu phân biệt hoa/thường

- **Found by test case:** TC-PRODLIST-101 · **Requirement:** FR-05 · **Severity:** Medium / P2

```
curl -G --data-urlencode "search=Áo" localhost:3000/api/products      # → 1 dòng
curl -G --data-urlencode "search=áo" localhost:3000/api/products      # → 0 dòng
curl -G --data-urlencode "search=Laptop" localhost:3000/api/products  # → 2 dòng
curl -G --data-urlencode "search=laptop" localhost:3000/api/products  # → 2 dòng
```

Expected: áo và Áo cho cùng kết quả — SUT là ứng dụng tiếng Việt, và chính nó đã không phân biệt hoa/thường với ASCII. **Actual:** `LIKE` của `SQLite` chỉ không phân biệt hoa/thường trong phạm vi **ASCII**; ký tự Unicode có dấu thì phân biệt. Người dùng gõ "áo" — cách gõ tự nhiên nhất — không tìm được sản phẩm nào.

Vì sao bug này đáng kể dù severity Medium: nó không phải lỗi bảo mật và không xuất hiện nếu chỉ test bằng dữ liệu tiếng Anh. Đây là loại lỗi mà bộ test do AI sinh từ spec bỏ sót hoàn toàn — xem `test-cases/api-01-products-search/extended.md`.

BUG-19 — [BUG][users] GET /api/users/me trả mật khẩu dạng plaintext (SEC-01)

- **Requirement:** SEC-01 — Mật khẩu không được lưu dưới dạng plaintext

- **Severity / Priority: Critical / P1** · Ngoài phạm vi 3 API được giao — phát hiện khi dựng setup login cho API-02/API-03, báo vì đề yêu cầu *"report any genuine bugs you find"*

```
curl localhost:3000/api/users/me -H "Authorization: Bearer $USER_TOKEN"
# → {"id":2,...,"password":"Test1234!","role":"user",...,"reset_token":null,...}
```

Actual: server.js:112–116 dùng `SELECT * FROM users` rồi `res.json(user)` → trả cả `password`, `reset_token`, `login_attempts`, `locked_until`. Mật khẩu là plaintext trong CSDL (server.js:20–30 insert thẳng, :32–51 so sánh trực tiếp), nên vi phạm SEC-01 ở **cả hai mặt**: lưu trữ và phơi bày.

Các bug còn lại — bằng chứng

BUG-02, 03, 04, 06, 07, 09, 10, 11, 12, 16, 17, 18 đều có **issue riêng** (#324, #325, #326, #328, #329, #331, #332, #333, #334, #338, #339, #340) và có khối *request* → *response* → *kết luận* riêng trong [verify-bugs-output.txt](#) (chạy lại: `bash bug-report/verify-bugs.sh <số>`), và mỗi bug map tới test case đã nêu ở bảng tổng hợp. Không có bug nào trong báo cáo này chỉ dựa trên việc đọc source.

Bug đã loại sau khi kiểm chứng

Ghi lại để không nhận vơ:

Giả thuyết ban đầu	Kết luận sau khi thử thật
Giỏ hàng của hai user không được cách ly (IDOR)	Không phải bug. <code>userCarts[req.user.id]</code> tách theo <code>id</code> trong JWT; TC-CART-030 xanh — giỏ user2 không thấy sản phẩm của user1
<code>POST /api/cart</code> thiếu kiểm token	Không phải bug. 401 khi không có header, 403 khi token rác (TC-CART-031...034 xanh) — SEC-02 đạt ở endpoint này
<code>'; DROP TABLE products--</code> xóa được bảng	Không đúng mức độ. <code>db.all()</code> chỉ chạy câu đầu; bảng còn nguyên (TC-PRODLIST-106 xanh). Mức khai thác thật là đọc dữ liệu + dò cấu trúc
<code>:id</code> chứa <code>1 OR 1=1</code> gây cập nhật hàng loạt	Không phải bug. <code>:id</code> đi qua parameterized query (server.js:181–187); TC-PRODUPD-025 xanh. Route này chỉ thiếu auth , không thiếu escape